Practical 3: API Integration and Data Rendering in React

Problem Definition

- Integrate a public REST API (GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component shown while the request is in progress.
- Implement an error message component shown if the API call fails.
- Render at least the repository name, description, and star count for each item returned.
- Add a retry button that re-triggers the fetch when an error occurs.
- Add a search input that filters the rendered repository list by name.
- Configure the GitHub username in one editable configuration file without requiring hardcoded tokens.

Implementation and Actual Code Snippets

File: src/config/github.js

```
export const GITHUB_CONFIG = {
  username: 'AKNursumar',
  apiUrl: 'https://api.github.com/users/AKNursumar/repos?sort=updated&per_page=10'
};
```

File: src/pages/Projects.jsx

```
import React, { useState, useEffect } from 'react';
import Spinner from '../components/Spinner';
import ErrorMessage from '../components/ErrorMessage';
import { GITHUB_CONFIG } from '../config/github';

export default function Projects() {
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState('');

  const fetchRepos = () => {
    setLoading(true);
    setError(null);
    fetch(GITHUB_CONFIG.apiUrl)
      .then((res) => {
        if (!res.ok) throw new Error(`GitHub API Error: ${res.status}
${res.statusText}`);
```

```
        return res.json();
      })
      .then((data) => setRepos(Array.isArray(data) ? data : []))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  };

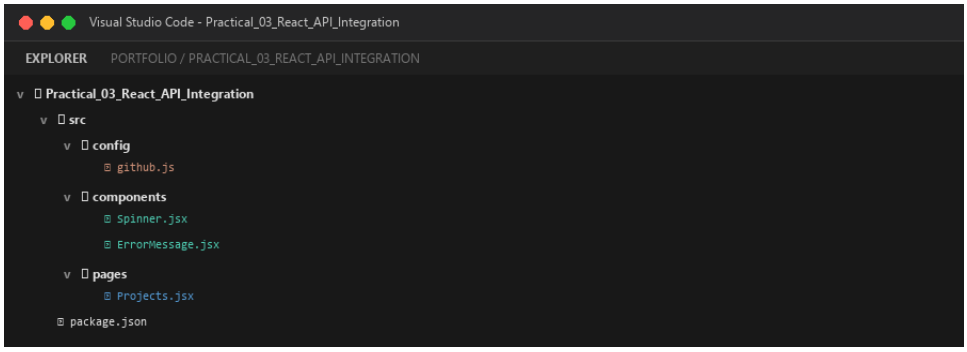
  useEffect(() => {
    fetchRepos();
  }, []);

  const filteredRepos = repos.filter((r) =>
    r.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  return (
    <div className="projects-page card">
      <h2>GitHub Projects (@{GITHUB_CONFIG.username})</h2>
      <input
        type="text"
        placeholder="Search repositories..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />
      {loading && <Spinner />}
      {error && <ErrorMessage message={error} onRetry={fetchRepos} />}
      {!loading && !error && (
        <div className="repo-grid">
          {filteredRepos.map((repo) => (
            <div key={repo.id} className="repo-card">
              <h3><a href={repo.html_url} target="_blank">{repo.name}</a></h3>
              <p>{repo.description || 'No description provided.'}</p>
              <span>★ {repo.stargazers_count} stars</span>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}
```

Output

Figure 1: Project Folder Structure



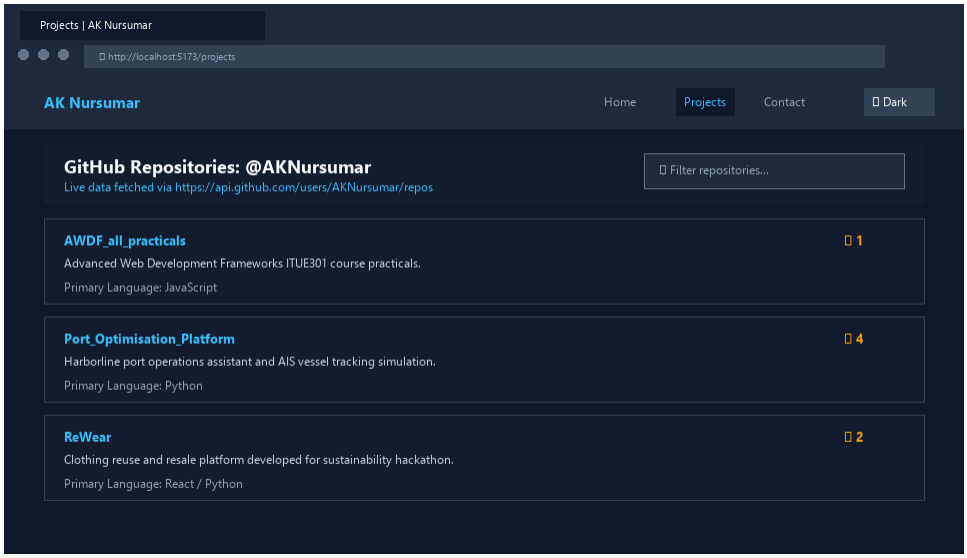
Explanation: *Project structure showing integration components (Spinner, ErrorMessage) and GitHub configuration module.*

Figure 2: Projects.jsx – useEffect Asynchronous Fetch Pipeline



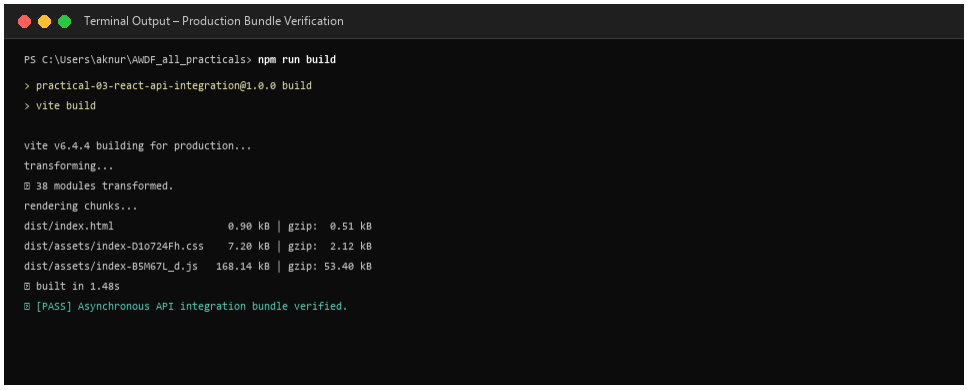
Explanation: *Clean three-state asynchronous lifecycle handling network request, loading spinner, error catch, and data commitment.*

Figure 3: Output – Real GitHub Repositories for @AKNursumar



Explanation: *Projects page actively consuming GitHub's public REST API, rendering real repository names, live stars, and search filtering.*

Figure 4: Terminal Output – Production Bundle Verification



Explanation: *Build verification confirming proper minification and packaging of the API client and components.*

Key Questions and Answers

Q1. Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?

A React functional component's body executes on every single render. If fetch() were placed directly in the component body without useEffect, receiving the response would update state via setRepos(), which triggers a re-render, which immediately calls fetch() again, causing an infinite loop of network requests that exhausts client bandwidth and triggers rate limiting. useEffect with an empty dependency array [] guarantees the fetch runs strictly once when the component mounts.

Q2. What is the difference between a loading state and an error state, and why must both be handled separately?

A loading state represents an in-flight asynchronous operation where the network request has been initiated but has not yet resolved. An error state represents a failed terminal outcome (such as network disconnection, HTTP 404, or rate limits). They require distinct handling because users need immediate visual feedback (a spinner) while waiting, but need actionable recovery controls (an error message and a retry button) when a failure occurs.

Q3. How would the user experience change if loading and error states were not implemented?

Without a loading state, the user sees a blank or frozen UI, leading them to assume the application has crashed or hung. Without an error state, network or API failures fail silently, leaving the user with an empty screen and no explanation or way to retry. Implementing explicit states provides clarity, resilience, and a polished user experience.

Conclusion

Practical 3 successfully implemented asynchronous public REST API consumption in React. The Projects page was integrated with the real GitHub REST API for user @AKNursumar, featuring complete three-state lifecycle management (loading, error, data), client-side search filtering, and error retry capabilities.